Practical No. 7

C++ Program: Priority and Round Robin Scheduling (Average WT and TAT with Gantt Chart)

1. AIM

To compute and print the average waiting time and average turnaround time, with Gantt chart, using Priority Scheduling (non-preemptive) and Round Robin Scheduling algorithms.

2. REQUIREMENTS

- Operating System: Ubuntu / Linux (or Windows)
- Language: C++
- Compiler: g++ (GCC)
- Text editor: Nano / Vim / VS Code
- Terminal to compile and run
- Hardware: computer, keyboard (input), monitor (output)

Assumptions: all processes arrive at time 0; smaller priority number = higher priority; equal priority runs in input order; Round Robin uses the given time quantum.

3. SHORT THEORY

CPU scheduling: the OS selects a process from the ready queue and gives it the CPU. It improves CPU utilization, reduces waiting time and gives fair execution.

- Priority Scheduling: CPU goes to the process with the highest priority (smallest number). Here it is non-preemptive: once started, a process runs until it finishes. Disadvantage: low-priority processes may starve.
- Round Robin (RR): preemptive. Each process gets the CPU for a fixed time called the time quantum. If not finished, it goes to the end of the ready queue. It is fair and good for time-sharing systems.
- Burst Time (BT): total CPU time needed by a process.
- Gantt Chart: shows the order of execution with start and end times.

Formula	Note
`TAT = Completion Time - Arrival Time`	Arrival = 0, so TAT = Completion Time.
`WT = TAT - Burst Time`	Time spent waiting in the queue.
`Average WT = Total WT / n`	n = number of processes.
`Average TAT = Total TAT / n`	n = number of processes.

4. CODE (file name: sched.cpp)

```
#include <iostream>
using namespace std;

int main() {
    int n, i, j, quantum, temp;
    int bt[20], priority[20], wt[20], tat[20];
    int pid[20], orig_bt[20];

    cout << "Enter number of processes: ";
    cin >> n;

    for (i = 0; i < n; i++) {
```

```
    pid[i] = i + 1;
    cout << "Enter burst time and priority of P" << i + 1 << ": ";
    cin >> bt[i] >> priority[i];
    orig_bt[i] = bt[i];      // keep original order for Round Robin
}

// -------- Priority Scheduling --------
// Sort by priority (smaller number = higher priority).
// If priorities are equal, keep input order (smaller pid first).
for (i = 0; i < n - 1; i++) {
    for (j = i + 1; j < n; j++) {
        if (priority[i] > priority[j] ||
           (priority[i] == priority[j] && pid[i] > pid[j])) {
            temp = priority[i]; priority[i] = priority[j]; priority[j] = temp;
            temp = bt[i];       bt[i] = bt[j];             bt[j] = temp;
            temp = pid[i];      pid[i] = pid[j];           pid[j] = temp;
        }
    }
}

wt[0] = 0;
for (i = 1; i < n; i++)
    wt[i] = wt[i - 1] + bt[i - 1];

float avgWT = 0, avgTAT = 0;
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
    avgWT += wt[i];
    avgTAT += tat[i];
}

cout << "\nPriority Scheduling\n";
cout << "Gantt Chart:\n0 ";
for (i = 0; i < n; i++)
    cout << "| P" << pid[i] << " | " << tat[i] << " ";

cout << "\n\nProcess\tWT\tTAT\n";
for (i = 0; i < n; i++)
    cout << "P" << pid[i] << "\t" << wt[i] << "\t" << tat[i] << endl;

cout << "Average Waiting Time = " << avgWT / n;
cout << "\nAverage Turnaround Time = " << avgTAT / n;

// -------- Round Robin Scheduling --------
int rem[20], time = 0;
avgWT = 0;
avgTAT = 0;

cout << "\n\nEnter time quantum: ";
cin >> quantum;

for (i = 0; i < n; i++)
    rem[i] = orig_bt[i];

cout << "\nRound Robin Scheduling\n";
cout << "Gantt Chart:\n0 ";

while (true) {
    bool done = true;
    for (i = 0; i < n; i++) {
        if (rem[i] > 0) {
            done = false;
            cout << "| P" << i + 1 << " | ";
            if (rem[i] > quantum) {
                time += quantum;
                rem[i] -= quantum;
```

```
                } else {
                    time += rem[i];
                    rem[i] = 0;
                    tat[i] = time;
                    wt[i] = tat[i] - orig_bt[i];
                }
                cout << time << " ";
            }
        }
        if (done)
            break;
    }

    cout << "\n\nProcess\tWT\tTAT\n";
    for (i = 0; i < n; i++) {
        avgWT += wt[i];
        avgTAT += tat[i];
        cout << "P" << i + 1 << "\t" << wt[i] << "\t" << tat[i] << endl;
    }

    cout << "Average Waiting Time = " << avgWT / n;
    cout << "\nAverage Turnaround Time = " << avgTAT / n << endl;
    return 0;
}
```

The slide version of this code has three problems that I fixed: (1) Round Robin was using the priority-sorted order, but it should use the original input order P1, P2, P3. I keep a copy of the burst times in orig_bt[]. (2) Priority Scheduling did not print a Gantt chart, but the problem asks for one. (3) The swap did not keep input order for equal priorities, so I added the pid[i] > pid[j] check.

5. HOW TO EXECUTE (step by step)

Command / Key	What it does
`Ctrl + Alt + T`	Open the Ubuntu terminal.
`sudo apt install g++`	Only if g++ is not installed. Check with: g++ --version
`cd ~`	Go to the home folder.
`nano sched.cpp`	Create the file, paste the code, save (Ctrl+O, Enter, Ctrl+X).
`g++ sched.cpp -o sched`	Compile. g++ is the C++ compiler, -o sched names the output program.
`./sched`	Run the program.
`(enter values)`	Number of processes, then burst time and priority of each (two numbers on one line), then the time quantum.

6. LINE-BY-LINE EXPLANATION

Code	Meaning
`#include <iostream>`	Adds the library for input (cin) and output (cout).
`using namespace std;`	Lets us write cout and cin directly instead of std::cout.
`int main() {`	Program starts here.
`int n, i, j, quantum, temp;`	n = processes, i and j = loop counters, quantum = time quantum, temp = helper for swapping.
`int bt[20], priority[20], wt[20], tat[20];`	Arrays for burst time, priority, waiting time and turnaround time.

Code	Meaning
`int pid[20], orig_bt[20];`	pid = process number. orig_bt = copy of burst times in original order (used by Round Robin).
`cin >> n;`	Reads the number of processes.
`pid[i] = i + 1;`	Gives process numbers 1, 2, 3, ...
`cin >> bt[i] >> priority[i];`	Reads burst time and priority together (two values separated by space).
`orig_bt[i] = bt[i];`	Saves the original burst time because bt[] will be re-ordered in the sorting.
`for (i...) for (j...) if (priority[i] > priority[j] ...)`	Sorting. Compares two processes; if the first has a bigger priority number (lower priority), they are swapped. If equal, the one with the bigger pid goes later.
`temp = ...; (three swaps)`	Swaps priority, burst time and pid together so each process stays with its own data.
`wt[0] = 0;`	First process in the sorted order does not wait.
`wt[i] = wt[i-1] + bt[i-1];`	Waiting time = previous waiting time + previous burst time.
`tat[i] = wt[i] + bt[i];`	Turnaround time = waiting time + burst time.
`avgWT += wt[i]; avgTAT += tat[i];`	Adds up totals. Later divided by n to get averages.
`cout << "| P" << pid[i] << " | " << tat[i];`	Prints the Gantt chart. tat[i] is the end time of that process.
`avgWT / n`	Average. avgWT is declared float so the answer has decimals.
`int rem[20], time = 0;`	rem = remaining burst time of each process. time = current clock.
`cin >> quantum;`	Reads the time quantum.
`rem[i] = orig_bt[i];`	Starts with the full burst time, in input order P1, P2, P3.
`while (true) {`	Keeps looping over all processes until all are finished.
`bool done = true;`	Assume all finished. Set to false if any process still has time left.
`if (rem[i] > 0)`	Only handle processes that are not finished.
`if (rem[i] > quantum)`	Process needs more than one quantum: run for quantum, add to time, reduce rem.
`else { time += rem[i]; ... }`	Process finishes in this turn: add the remaining time, set rem to 0, record tat[i] = time and wt[i] = tat[i] - orig_bt[i].
`cout << time << " ";`	Prints the end time of this slice in the Gantt chart.
`if (done) break;`	If no process is left, exit the while loop.
`return 0;`	Ends the program.

7. MEANING OF SYMBOLS

Symbol	Use
`cout <<`	Output. Sends text or values to the screen. << is the insertion operator.
`cin >>`	Input. Reads values from the keyboard. >> is the extraction operator.

Symbol	Use
`endl`	New line (and flushes the output).
`\n \t`	New line and tab space.
`[ ]`	Array index. bt[0] is the first element.
`+= -=`	Add / subtract and store. x += 2 means x = x + 2.
`|| &&`	OR and AND. Used to combine conditions.
`> ==`	Greater than, and equal to. (Single = assigns, == compares.)
`bool`	Data type that holds true or false.
`// ...`	Comment. Ignored by the compiler.
`{ }`	Start and end of a block.

8. SAMPLE INPUT AND OUTPUT

Input: 3 processes, quantum = 2. P1: BT 5, Priority 2. P2: BT 3, Priority 1. P3: BT 4, Priority 3.

Gantt chart	Calculation
`Priority: 0 | P2 | 3 | P1 | 8 | P3 | 12`	WT: P2=0, P1=3, P3=8. TAT: 3, 8, 12. Avg WT = 11/3 = 3.67. Avg TAT = 23/3 = 7.67
`Round Robin (q=2): 0 | P1 | 2 | P2 | 4 | P3 | 6 | P1 | 8 | P2 | 9 | P3 | 11 | P1 | 12`	WT: P1=7, P2=6, P3=7. TAT: 12, 9, 11. Avg WT = 20/3 = 6.67. Avg TAT = 32/3 = 10.67

The sample output printed on the assignment slide does not match its own sample input, so use the values above, which I got by compiling and running the program. Your terminal prints 3.66667 and 6.66667 because cout shows 6 digits by default.

9. CONCLUSION

Thus, we have implemented non-preemptive Priority Scheduling and Round Robin Scheduling in C++, displayed the Gantt charts and calculated the waiting time, turnaround time and their averages for both algorithms.

10. QUICK VIVA ANSWERS

- CPU scheduling: choosing which process from the ready queue gets the CPU. It is important for good CPU utilization, less waiting time and fairness.
- Priority Scheduling: the process with the highest priority (smallest number here) runs first. E.g. priorities P1=2, P2=1, P3=3 run as P2, P1, P3.
- Preemptive vs non-preemptive priority: in preemptive, a newly arrived higher-priority process can interrupt the running one. In non-preemptive, the running process finishes first.
- Round Robin: each process gets the CPU for one time quantum, then goes to the back of the queue. A very large quantum behaves like FCFS; a very small quantum causes too many context switches.
- Gantt chart: a timeline of which process runs when. It is used to find completion, waiting and turnaround times.
- Waiting time: time spent in the ready queue (WT = TAT - BT). Turnaround time: total time from arrival to completion (TAT = Completion - Arrival).

- Averages: add all waiting (or turnaround) times and divide by the number of processes.